☸ Kubernetes - Complete Hands-On Guide

Table of Contents

1. Introduction to Containers & Orchestration
2. Kubernetes Architecture
3. Setting Up Kubernetes Cluster
4. Kubectl Commands
5. Deploying Applications
6. Exposing Services
7. Scaling Applications
8. Updating & Rollback
9. YAML Configuration
10. Hands-On Labs
11. Quick Reference

1. Introduction to Containers & Orchestration

What is a Container?

Containers provide a standard way to package your application's code, configurations, and dependencies into a single object. They share an operating system installed on the server and run as resource-isolated processes, ensuring quick, reliable, and consistent deployments regardless of environment.

Key Benefits of Containers:

- **Portability:** Run anywhere (local, cloud, on-premise)
- **Isolation:** Applications run in isolated environments
- **Efficiency:** Lightweight compared to virtual machines
- **Speed:** Quick startup and shutdown times

What is Container Orchestration?

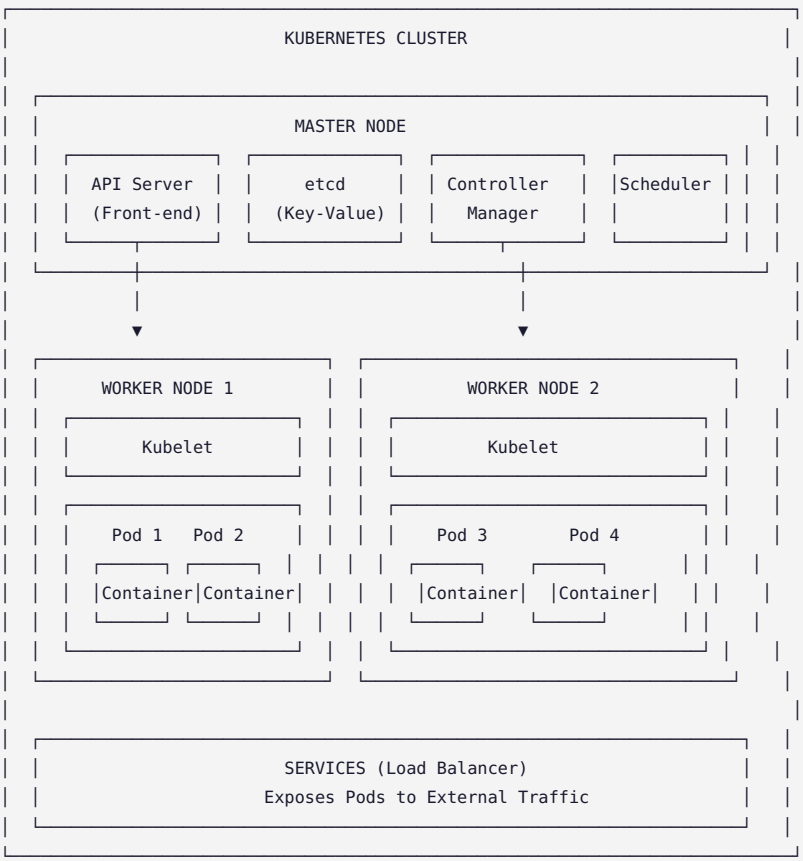
Container orchestration automates the deployment, scaling, and management of containerized applications. Kubernetes is the leading orchestration platform, inspired by Google's internal project called Borg.

Why Kubernetes?

- **Planet Scale:** Designed on principles that allow Google to run billions of containers weekly
- **Never Outgrow:** Flexibility grows with you from local testing to global enterprise
- **Run Anywhere:** Open source, works on-premises, hybrid, or public cloud

2. Kubernetes Architecture

Architecture Overview



Components Explained

Master Components

Component	Description	Purpose
kube-apiserver	Front-end for control plane	Validates and configures objects, handles REST operations
etcd	Distributed key-value store	Stores all cluster data, cluster state
kube-controller-manager	Runs controller threads	Handles routine tasks (node, replication, endpoints)
kube-scheduler	Pod assignment	Assigns pods to nodes based on resource requirements
Cluster DNS	DNS service	Serves DNS records for Kubernetes services

Node Components

Component	Description	Purpose
Kubelet	Node agent	Manages containers, communicates with master
kube-proxy	Network proxy	Maintains network rules, load balancing
Container Runtime	Container engine	Runs containers (Docker, containerd, etc.)

Key Kubernetes Concepts

POD

- Smallest deployable unit
- Collection of one or more containers
- Shares network and storage
- Has unique IP address

REPLICATION CONTROLLER

- Ensures specified number of pods running
- Self-healing capabilities
- Manages pod lifecycle

SERVICE

- Load balancing across pods
- Stable IP address
- Service discovery via DNS

---

3. Setting Up Kubernetes Cluster

Option 1: Minikube (Local Development)

> **Prerequisites:**
>
> - VT-x or AMD-v virtualization enabled in BIOS
> - Hypervisor installed (VirtualBox, Hyper-V, etc.)
> - Docker installed

Installation Steps:

```bash
# 1. Install Docker
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo apt-key add -
sudo add-apt-repository "deb [arch=amd64] https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable"
sudo apt-get update
sudo apt-get install -y docker-ce
sudo systemctl status docker

# 2. Install kubectl
curl -LO "https://dl.k8s.io/release/$(curl -L -s https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"
chmod +x kubectl
sudo mv kubectl /usr/local/bin/

# 3. Install Minikube
curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube-linux-amd64
```

```
sudo install minikube-linux-amd64 /usr/local/bin/minikube

# 4. Start Minikube
minikube start --driver=virtualbox

# 5. Verify installation
kubectl version
minikube status
```

Output Example:

```
minikube v0.34.1 on linux (amd64)
Configuring local host environment ...
Creating none VM (CPUs=2, Memory=2048MB, Disk=20000MB)
"minikube" IP address is 172.17.0.18
Configuring Docker as the container runtime ...
Done! Thank you for using minikube!
```

Option 2: kubeadm (Multi-Node Cluster)

On Master Node:

BASH

```
# Initialize the cluster with pod network CIDR
sudo kubeadm init --pod-network-cidr=10.244.0.0/16

# Set up kubectl for regular user
mkdir -p $HOME/.kube
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config

# Install Flannel pod network
kubectl apply -f https://raw.githubusercontent.com/coreos/flannel/v0.9.1/Documentation/kube-flannel.yml

# Get join command from the init output
# Example:
# kubeadm join 172.31.19.4:6443 --token 515716.5146fe2f99ae710c \
# --discovery-token-ca-cert-hash sha256:e94a50c90fc84843cfb79d800aa9d9a455bb3c67cecc63d12475aee3763d6d1
```

On Worker Nodes:

BASH

```
# Join the cluster using the token from master
kubeadm join <MASTER_IP>:6443 --token <TOKEN> --discovery-token-ca-cert-hash <HASH>

# On Master, verify nodes joined
kubectl get nodes
```

Output Example:

```
NAME               STATUS   ROLES    AGE   VERSION
ip-172-31-19-4     Ready    master   12m   v1.13.3
ip-172-31-46-252   Ready    <none>   5m    v1.13.3
```

Option 3: AWS EC2 Cluster Setup

☁ AWS EC2 INSTANCE LAUNCH WIZARD

Step	Configuration
Step 1: Choose AMI	Ubuntu Server 18.04 LTS (or 20.04 LTS) • 64-bit (x86)

Step 2: Choose Instance Type	Master: t2.medium (2 vCPU, 4 GB RAM) • Worker: t2.medium
Step 3: Configure Instance	Number: 2 (1 master, 1 worker) • Auto-assign Public IP: Enable
Step 4: Add Storage	Root volume: 20 GB (minimum) • Type: gp2
Step 5: Add Tags	Key: Name → Value: k8s-master • Key: Name → Value: k8s-worker
Step 6: Configure Security Group	SSH (22), HTTPS (443), HTTP (80), API (6443), NodePort (30000-32767)
Step 7: Review and Launch	Key pair: Create new or select existing • Launch instances

4. Kubectl Commands

Basic Commands

BASH

```
# Check version and connectivity
kubectl version

# Get cluster information
kubectl cluster-info

# Get cluster info dump (for debugging)
kubectl cluster-info dump
```

Resource Management Commands

BASH

```
# Get resources
kubectl get nodes
kubectl get pods
kubectl get deployments
kubectl get services
kubectl get replicasets
kubectl get all

# Describe resources (detailed view)
kubectl describe pod <pod-name>
kubectl describe deployment <deployment-name>
kubectl describe service <service-name>

# Get resources with custom output
kubectl get pods -o wide                    # More details
kubectl get pods -o yaml                    # YAML format
kubectl get pods -o json                    # JSON format
kubectl get pods --watch                    # Watch for changes

# Get resources by labels
kubectl get pods -l app=nginx
kubectl get pods -l 'app in (nginx,web)'
kubectl get pods -l app!=nginx
```

Logging and Debugging

BASH

```
# View pod logs
kubectl logs <pod-name>
kubectl logs <pod-name> -f                  # Follow logs
kubectl logs <pod-name> --tail=50           # Last 50 lines
```

```
# Execute commands in container
kubectl exec -it <pod-name> -- /bin/bash
kubectl exec -it <pod-name> -- sh
kubectl exec <pod-name> -- ls -la

# Port forwarding
kubectl port-forward pod/<pod-name> 8080:80
kubectl port-forward deployment/<deployment-name> 8080:80
```

Create and Delete Resources

BASH

```
# Create resources
kubectl create -f <filename.yaml>
kubectl apply -f <filename.yaml>           # Recommended (declarative)

# Delete resources
kubectl delete pod <pod-name>
kubectl delete deployment <deployment-name>
kubectl delete service <service-name>
kubectl delete -f <filename.yaml>

# Edit resource
kubectl edit deployment <deployment-name>
kubectl edit service <service-name>

# Patch resource
kubectl patch deployment <name> -p '{"spec":{"replicas":3}}'
```

---

5. Deploying Applications

Imperative Approach (Quick Start)

BASH

```
# Basic deployment
kubectl run myapp --image=nginx --port=80

# Deployment with multiple replicas
kubectl run myapp --image=nginx --replicas=3 --port=80

# Deployment with environment variables
kubectl run myapp --image=nginx --env="ENV=production"

# Deployment with labels
kubectl run myapp --image=nginx --labels="app=myapp,tier=frontend"

# Example: Hello World application
kubectl run hello-world --replicas=2 \
  --labels="run=load-balancer-example" \
  --image=gcr.io/google-samples/node-hello:1.0 \
  --port=8080
```

Declarative Approach (Recommended)

Create `deployment.yaml`:

YAML

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
  labels:
    app: nginx
    tier: frontend
spec:
```

```
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
        tier: frontend
    spec:
      containers:
      - name: nginx
        image: nginx:1.21
        ports:
        - containerPort: 80
        env:
        - name: ENVIRONMENT
          value: production
        resources:
          requests:
            memory: "64Mi"
            cpu: "250m"
          limits:
            memory: "128Mi"
            cpu: "500m"
```

BASH

```
# Deploy the application
kubectl apply -f deployment.yaml

# Check deployment status
kubectl get deployments
kubectl describe deployment nginx-deployment
kubectl get pods
```

Application Deployment Flow



---

6. Exposing Services

Service Types

SERVICE TYPES

CLUSTERIP

- Internal cluster IP only
- Accessible only within the cluster
- Default service type
- Example: Database services

NODEPORT

- Exposes service on static port on each node
- Accessible via NodeIP:NodePort
- NodePort range: 30000-32767
- Example: Web applications for testing

LOADBALANCER

- Uses cloud provider's load balancer
- Creates NodePort and ClusterIP automatically
- External IP assigned by cloud provider
- Example: Production web applications

Expose Service - Imperative

```bash
# Expose as NodePort
kubectl expose deployment nginx-deployment \
  --type=NodePort \
  --name=nginx-service \
  --port=80 \
  --target-port=80

# Expose as ClusterIP (default)
kubectl expose deployment nginx-deployment \
  --type=ClusterIP \
  --name=nginx-clusterip \
  --port=80

# Expose with specific NodePort
kubectl expose deployment nginx-deployment \
  --type=NodePort \
  --name=nginx-service \
  --port=80 \
  --node-port=30008
```

Complete Deployment + Service Example

Create `app-deployment.yaml`:

```yaml
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
  labels:
    app: nginx
```

```yaml
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: nginx:1.21
        ports:
        - containerPort: 80
---
apiVersion: v1
kind: Service
metadata:
  name: nginx-service
spec:
  type: NodePort
  selector:
    app: nginx
  ports:
  - protocol: TCP
    port: 80
    targetPort: 80
    nodePort: 30008
```

```bash
kubectl apply -f app-deployment.yaml

# Access the application
kubectl get nodes -o wide          # Get node IPs
curl http://<Node-IP>:30008        # Access via NodePort
```

---

7. Scaling Applications

Imperative Scaling

```bash
# Scale deployment to specific number
kubectl scale deployment nginx-deployment --replicas=5

# Scale to 2 replicas
kubectl scale deployment nginx-deployment --replicas=2

# Scale multiple deployments
kubectl scale deployment nginx-deployment web-deployment --replicas=3

# Check scaling status
kubectl get deployments
kubectl get pods
kubectl get replicasets
```

Declarative Scaling

Update `deployment.yaml`:

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
spec:
```

```
  replicas: 5  # Change this value
  # ... rest of configuration
```

```
kubectl apply -f deployment.yaml
```
BASH

Horizontal Pod Autoscaler (HPA)

```
# Create autoscaler based on CPU
kubectl autoscale deployment nginx-deployment \
  --min=2 --max=10 --cpu-percent=50

# Check HPA status
kubectl get hpa
kubectl describe hpa nginx-deployment
```
BASH

---

8. Updating & Rollback

Rolling Update

```
# Update image version
kubectl set image deployment/nginx-deployment \
  nginx=nginx:1.22

# Check rollout status
kubectl rollout status deployment/nginx-deployment

# Check rollout history
kubectl rollout history deployment/nginx-deployment

# Pause/resume rollout
kubectl rollout pause deployment/nginx-deployment
kubectl rollout resume deployment/nginx-deployment
```
BASH

Rollback

```
# Rollback to previous version
kubectl rollout undo deployment/nginx-deployment

# Rollback to specific revision
kubectl rollout undo deployment/nginx-deployment --to-revision=2

# Check rollout status after rollback
kubectl rollout status deployment/nginx-deployment
```
BASH

Update Strategies

ROLLING UPDATE (Default)

- Gradual replacement of pods
- Zero downtime
- Configurable maxSurge and maxUnavailable

RECREATE

- Terminate all pods first
- Create new pods after termination
- Downtime expected

BLUE-GREEN (Manual)

- Deploy new version alongside old
- Switch traffic when ready
- Quick rollback by switching back

---

9. YAML Configuration

YAML Basics

YAML STRUCTURES

MAPS (Key-Value Pairs)

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx-pod
  namespace: default
  labels:
    app: nginx
    tier: frontend
```

LISTS (Sequences)

```
containers:
  - name: nginx
    image: nginx
  - name: sidecar
    image: busybox
```

Pod YAML

YAML

```
apiVersion: v1
kind: Pod
metadata:
  name: busybox-pod
  namespace: default
  labels:
    app: busybox
    environment: dev
spec:
  containers:
  - name: busybox
    image: busybox
    command:
    - sleep
```

```
    - "3600"
    env:
    - name: MY_ENV
      value: "value"
    resources:
      requests:
        memory: "64Mi"
        cpu: "250m"
      limits:
        memory: "128Mi"
        cpu: "500m"
  restartPolicy: Always
```

Deployment YAML

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
  namespace: default
  labels:
    app: nginx
    tier: frontend
spec:
  replicas: 3
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 25%
      maxUnavailable: 25%
  selector:
    matchLabels:
      app: nginx
      tier: frontend
  template:
    metadata:
      labels:
        app: nginx
        tier: frontend
    spec:
      containers:
      - name: nginx
        image: nginx:1.21
        ports:
        - containerPort: 80
          name: http
        livenessProbe:
          httpGet:
            path: /
            port: 80
          initialDelaySeconds: 30
          periodSeconds: 10
        readinessProbe:
          httpGet:
            path: /
            port: 80
          initialDelaySeconds: 5
          periodSeconds: 5
        resources:
          requests:
            memory: "64Mi"
            cpu: "250m"
          limits:
            memory: "128Mi"
            cpu: "500m"
```

Service YAML

```yaml
apiVersion: v1
kind: Service
```

```
metadata:
  name: nginx-service
  namespace: default
  labels:
    app: nginx
spec:
  type: NodePort
  selector:
    app: nginx
  ports:
  - name: http
    port: 80
    targetPort: 80
    nodePort: 30008
    protocol: TCP
```

---

10. Hands-On Labs

Lab 1: Create Your First Pod

Objective: Create and manage a simple pod

Step 1: Create Pod YAML

YAML

```yaml
# single-pod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: nginx-pod
  labels:
    app: nginx
spec:
  containers:
  - name: nginx
    image: nginx:alpine
    ports:
    - containerPort: 80
```

Step 2: Deploy and Verify

BASH

```bash
# Create pod
kubectl apply -f single-pod.yaml

# Verify pod status
kubectl get pods
kubectl describe pod nginx-pod

# Get pod details
kubectl get pod nginx-pod -o wide
kubectl get pod nginx-pod -o yaml

# View logs
kubectl logs nginx-pod
```

Step 3: Access the Pod

BASH

```bash
# Port forward to access pod
kubectl port-forward pod/nginx-pod 8080:80

# In another terminal
curl http://localhost:8080

# Or access directly from pod
kubectl exec -it nginx-pod -- /bin/bash
```

Step 4: Clean Up

```bash
kubectl delete -f single-pod.yaml
kubectl delete pod nginx-pod
```

Lab 2: Deploy and Scale Nginx Application

Objective: Create a deployment with multiple replicas and scale

Step 1: Create Deployment

```yaml
# nginx-deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
  labels:
    app: nginx
spec:
  replicas: 2
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: nginx:latest
        ports:
        - containerPort: 80
```

```bash
kubectl apply -f nginx-deployment.yaml
kubectl get deployments
kubectl get pods
```

Step 2: Scale the Application

```bash
# Scale to 5 replicas
kubectl scale deployment nginx-deployment --replicas=5

# Verify scaling
kubectl get pods
kubectl get replicasets

# Scale to 3 replicas
kubectl scale deployment nginx-deployment --replicas=3
```

Step 3: Create Service

```yaml
# nginx-service.yaml
apiVersion: v1
kind: Service
metadata:
  name: nginx-service
spec:
  type: NodePort
  selector:
    app: nginx
  ports:
  - protocol: TCP
    port: 80
```

```
    targetPort: 80
    nodePort: 30003
```

```bash
kubectl apply -f nginx-service.yaml

# Check service
kubectl get services
kubectl describe service nginx-service

# Access the service
kubectl get nodes -o wide
curl http://<NODE_IP>:30003
```

Lab 3: Hello World Application

Objective: Deploy, expose, scale, and update a complete application

Step 1: Deploy Hello World

```bash
# Create deployment
kubectl run hello-world \
  --replicas=2 \
  --labels="run=load-balancer-example" \
  --image=gcr.io/google-samples/node-hello:1.0 \
  --port=8080

# Check deployment
kubectl get deployments hello-world
kubectl describe deployments hello-world
kubectl get pods
```

Step 2: Expose as Service

```bash
# Expose as NodePort
kubectl expose deployment hello-world \
  --type=NodePort \
  --name=hello-service

# Get service information
kubectl get services
kubectl describe services hello-service

# Access application
curl http://<NODE_IP>:<NODE_PORT>
```

Step 3: Scale Application

```bash
# Scale to 4 replicas
kubectl scale deployment hello-world --replicas=4

# Verify scaling
kubectl get deployments
kubectl get pods
kubectl get replicasets
```

Step 4: Update Application

```bash
# Update to version 2.0
kubectl set image deployment/hello-world \
  hello-world=gcr.io/google-samples/node-hello:2.0

# Check rollout status
kubectl rollout status deployment/hello-world
```

```
# Check history
kubectl rollout history deployment/hello-world
```

Step 5: Rollback Update

```bash
# Rollback to previous version
kubectl rollout undo deployment/hello-world

# Check rollback status
kubectl rollout status deployment/hello-world
```

Step 6: Clean Up

```bash
kubectl delete service hello-service
kubectl delete deployment hello-world
```

Lab 4: Multi-Container Pod

Objective: Create and manage a pod with multiple containers

Step 1: Create Multi-Container Pod

```yaml
# multi-container.yaml
apiVersion: v1
kind: Pod
metadata:
  name: webserver
spec:
  volumes:
  - name: shared-data
    emptyDir: {}
  containers:
  - name: nginx
    image: nginx
    volumeMounts:
    - name: shared-data
      mountPath: /usr/share/nginx/html
  - name: updater
    image: busybox
    volumeMounts:
    - name: shared-data
      mountPath: /data
    command:
    - sh
    - -c
    - |
      while true; do
        echo "$(date): Hello from sidecar" > /data/index.html
        sleep 5
      done
```

```bash
kubectl apply -f multi-container.yaml
kubectl get pods
kubectl describe pod webserver
```

Step 2: Access Both Containers

```bash
# Access nginx container
kubectl exec -it webserver -c nginx -- /bin/bash

# Access updater container
kubectl exec -it webserver -c updater -- /bin/sh

# View logs from both containers
```

```
kubectl logs webserver -c nginx
kubectl logs webserver -c updater
```

Step 3: Test Shared Volume

```bash
# Port forward nginx
kubectl port-forward pod/webserver 8080:80

# Access web page
curl http://localhost:8080

# Should show: "timestamp: Hello from sidecar"
```

Lab 5: Complete Microservices Application

Objective: Deploy a complete multi-tier application

Step 1: Create MySQL Deployment

```yaml
# mysql-deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mysql
spec:
  selector:
    matchLabels:
      app: mysql
  template:
    metadata:
      labels:
        app: mysql
    spec:
      containers:
      - name: mysql
        image: mysql:5.7
        env:
        - name: MYSQL_ROOT_PASSWORD
          value: password
        - name: MYSQL_DATABASE
          value: mydb
        ports:
        - containerPort: 3306
---
apiVersion: v1
kind: Service
metadata:
  name: mysql-service
spec:
  selector:
    app: mysql
  ports:
  - port: 3306
    targetPort: 3306
```

Step 2: Create Web Application Deployment

```yaml
# web-deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-app
spec:
  replicas: 3
  selector:
    matchLabels:
      app: web
```

```
  template:
    metadata:
      labels:
        app: web
    spec:
      containers:
      - name: web
        image: nginx:latest
        ports:
        - containerPort: 80
---
apiVersion: v1
kind: Service
metadata:
  name: web-service
spec:
  type: NodePort
  selector:
    app: web
  ports:
  - port: 80
    targetPort: 80
    nodePort: 30004
```

Step 3: Deploy the Complete Application

```bash
# Deploy MySQL
kubectl apply -f mysql-deployment.yaml

# Deploy Web
kubectl apply -f web-deployment.yaml

# Verify all resources
kubectl get pods
kubectl get deployments
kubectl get services
```

Step 4: Scale Web Application

```bash
# Scale web to 5 replicas
kubectl scale deployment web-app --replicas=5

# Verify scaling
kubectl get pods
kubectl get deployments web-app
```

Step 5: Update Web Application

```bash
# Update to new version
kubectl set image deployment/web-app web=nginx:1.25

# Check rollout status
kubectl rollout status deployment/web-app

# Rollback if needed
kubectl rollout undo deployment/web-app
```

Step 6: Clean Up

```bash
kubectl delete -f mysql-deployment.yaml
kubectl delete -f web-deployment.yaml
```

---

11. Quick Reference

Common Commands Cheat Sheet

Action	Command
Create Resources	
Create from YAML	`kubectl apply -f <file.yaml>`
Create deployment	`kubectl run <name> --image=<image>`
Create service	`kubectl expose deployment <name> --port=80`
Get Resources	
Get all resources	`kubectl get all`
Get pods	`kubectl get pods`
Get deployments	`kubectl get deployments`
Get services	`kubectl get services`
Get nodes	`kubectl get nodes`
Get with more details	`kubectl get pods -o wide`
Get with YAML output	`kubectl get pods -o yaml`
Describe Resources	
Describe pod	`kubectl describe pod <name>`
Describe deployment	`kubectl describe deployment <name>`
Describe service	`kubectl describe service <name>`
Scale	
Scale deployment	`kubectl scale deployment <name> --replicas=N`
Autoscale	`kubectl autoscale deployment <name> --min=N --max=N`
Update	
Update image	`kubectl set image deployment/<name> <container>=<image>`
Rollout status	`kubectl rollout status deployment/<name>`
Rollout history	`kubectl rollout history deployment/<name>`
Rollback	`kubectl rollout undo deployment/<name>`
Logs and Debug	
View logs	`kubectl logs <pod-name>`
Follow logs	`kubectl logs <pod-name> -f`
Execute in pod	`kubectl exec -it <pod-name> -- /bin/bash`
Port forward	`kubectl port-forward pod/<pod-name> 8080:80`
Delete Resources	
Delete pod	`kubectl delete pod <name>`
Delete deployment	`kubectl delete deployment <name>`
Delete service	`kubectl delete service <name>`

Delete from file	`kubectl delete -f <file.yaml>`

YAML Quick Reference

Field	Description
`apiVersion`	Kubernetes API version (v1, apps/v1, etc.)
`kind`	Resource type (Pod, Deployment, Service, etc.)
`metadata`	Resource metadata (name, labels, annotations)
`spec`	Desired state specification
`status`	Current state (automatically managed)

Useful Tips

- **Always use** `kubectl apply` instead of `create` for declarative management
- **Use** `kubectl get all` to see all resources in a namespace
- **Use** `-o wide` for additional details in output
- **Use** `--dry-run` to test commands without applying changes
- **Use** `kubectl explain` to understand resource fields

```bash
# Examples
kubectl explain pod
kubectl explain deployment.spec
kubectl explain service.spec
```

Appendices

Appendix A: Installation Verification Commands

```bash
# Verify kubectl
kubectl version --client
kubectl cluster-info

# Verify Minikube
minikube status
minikube ip

# Verify kubeadm
kubeadm version
kubeadm init --dry-run
```

Appendix B: Troubleshooting Common Issues

```bash
# Pod stuck in Pending state
kubectl describe pod <pod-name>
kubectl get events --sort-by='.lastTimestamp'

# Pod stuck in CrashLoopBackOff
kubectl logs <pod-name>
```

```
kubectl logs <pod-name> --previous

# Node not ready
kubectl describe node <node-name>
kubectl get events -n kube-system

# Service not accessible
kubectl describe service <service-name>
kubectl get endpoints <service-name>
kubectl exec -it <pod-name> -- curl <service-name>
```

Appendix C: Useful Resources

- Official Documentation: https://kubernetes.io/docs/
- Kubectl Cheat Sheet: https://kubernetes.io/docs/reference/kubectl/cheatsheet/
- API Reference: https://kubernetes.io/docs/reference/kubernetes-api/
- Minikube Documentation: https://minikube.sigs.k8s.io/docs/

Summary

KUBERNETES JOURNEY

1. SETUP
├── Install Docker, kubectl, minikube/kubeadm
└── Create cluster (master + nodes)

2. DEPLOY
├── Create Pod (single container)
├── Create Deployment (manage replicas)
└── Create Service (expose application)

3. MANAGE
├── Scale (increase/decrease replicas)
├── Update (new image version)
└── Rollback (undo updates)

4. MONITOR
├── Logs (kubectl logs)
├── Describe (kubectl describe)
└── Events (kubectl get events)

☸ Kubernetes - Complete Hands-On Guide

End of Document